

The Mathematics of Schmidhuber Architectures

Miquel Noguer i Alonso
Artificial Intelligence Finance Institute

January 19, 2025

Abstract

We provide a comprehensive, chronologically ordered survey of key architectures introduced by Jürgen Schmidhuber, along with their theoretical underpinnings, essential mathematical formulations, and practical implementation outlines. Our exposition spans early self-referential meta-learning systems, fundamental recurrent neural networks, gated variants such as LSTM, and more advanced frameworks like Fast Weight Programmers, Gödel Machines, and PowerPlay. Where possible, we present relevant theorems, their proofs, and details on convergence, stability, and optimization strategies. This unified overview underscores the enduring impact of Schmidhuber’s ideas on the foundations of deep learning and artificial intelligence.

1 Introduction

This document provides a comprehensive and chronological overview of key architectures developed by Jürgen Schmidhuber. Each section includes theoretical foundations, rigorous mathematical frameworks, theorems, proofs, and detailed implementation algorithms. References to the original works are included to guide readers interested in further exploration.

2 Self-Referential Learning (1987, 1993)

2.1 Theoretical Foundation

Self-Referential Learning [Schmidhuber, 1987, 1993] explores meta-learning systems that can modify their own learning algorithms, enabling self-improvement over time. This is achieved through a meta-learning algorithm that updates the base learning algorithm based on performance feedback.

2.2 Mathematical Framework

Definition 1 (Self-Referential System). *A self-referential system $\mathcal{S}$ is defined as:*

$$\mathcal{S} = (A, M, \phi),$$

where:

- A : Learning algorithm, represented as a parameterized function A_θ .
- M : Meta-learning algorithm, represented as a function M_ϕ .
- ϕ : Update rule for A , typically a gradient-based optimization step.

Theorem 1 (Self-Improvement Convergence). *For a bounded meta-learning update rule ϕ , the system converges to an optimal learning algorithm:*

$$\lim_{t \rightarrow \infty} \|A_t - A^*\|_F = 0,$$

where A^* is the optimal learning algorithm.

Proof. 1. The meta-learning update at time t is given by:

$$A_t = \phi(A_{t-1}, M_{t-1}).$$

2. By the contraction mapping theorem, if ϕ is a contraction mapping with Lipschitz constant $\gamma < 1$, then:

$$\|\phi(A_{t-1}, M_{t-1}) - \phi(A^*, M^*)\|_F \leq \gamma \|A_{t-1} - A^*\|_F.$$

3. Iterating this inequality yields:

$$\|A_t - A^*\|_F \leq \gamma^t \|A_0 - A^*\|_F.$$

4. As $t \rightarrow \infty$, $\gamma^t \rightarrow 0$, and thus $\|A_t - A^*\|_F \rightarrow 0$.

□

2.3 Implementation Details

• Training Algorithm:

- 1: Initialize learning algorithm A_0 and meta-learning algorithm M_0 .
- 2: **while** not converged **do**
- 3: **for** each task $\mathcal{T}_i$ **do**
- 4: Evaluate performance of A_t on $\mathcal{T}_i$.
- 5: Compute meta-gradient: $\nabla_\phi \mathcal{L}(A_t, \mathcal{T}_i)$.
- 6: Update meta-learning algorithm: $M_t = M_{t-1} - \eta \nabla_\phi \mathcal{L}$.
- 7: Update learning algorithm: $A_t = \phi(A_{t-1}, M_t)$.
- 8: **end for**
- 9: **end while**

- **Optimization Techniques:**

- Gradient-based meta-learning (e.g., MAML).
- Evolutionary strategies for non-differentiable updates.
- Regularization to prevent overfitting in the meta-learning phase.

3 Recurrent Neural Networks (RNNs) (1990)

3.1 Theoretical Foundation

Recurrent Neural Networks (RNNs) [Schmidhuber, 1990] represent a fundamental architecture for processing sequential data. They maintain a hidden state that captures temporal dependencies, enabling the network to learn patterns across time steps.

3.2 Mathematical Framework

Definition 2 (RNN System). *An RNN system $\mathcal{R}$ is defined as:*

$$\mathcal{R} = (H, U, W, V, f),$$

where:

- H : Hidden state space.
- U : Input-to-hidden weight matrix.
- W : Hidden-to-hidden weight matrix.
- V : Hidden-to-output weight matrix.
- f : Activation function.

Theorem 2 (RNN Stability). *For a Lipschitz continuous activation function f with Lipschitz constant L_f , the RNN hidden state converges to a fixed point if:*

$$\|W\|_2 < \frac{1}{L_f}.$$

Proof. 1. The hidden state update equation is:

$$h_t = f(Ux_t + Wh_{t-1}).$$

2. Consider two consecutive hidden states:

$$\|h_t - h_{t-1}\| = \|f(Ux_t + Wh_{t-1}) - f(Ux_{t-1} + Wh_{t-2})\|.$$

3. By Lipschitz continuity:

$$\|h_t - h_{t-1}\| \leq L_f \|W\|_2 \|h_{t-1} - h_{t-2}\|.$$

4. If $L_f \|W\|_2 < 1$, the sequence $\{h_t\}$ is a contraction mapping and converges. $\square$

3.3 Implementation Details

- **Training Algorithm:**

```
1: Initialize parameters  $U$ ,  $W$ , and  $V$ .
2: while not converged do
3:   for each input sequence  $(x_1, \dots, x_T)$  do
4:     Initialize hidden state  $h_0$ .
5:     for each time step  $t$  do
6:       Compute hidden state:  $h_t = f(Ux_t + Wh_{t-1})$ .
7:       Compute output:  $y_t = Vh_t$ .
8:       Compute loss:  $\mathcal{L}(y_t, \hat{y}_t)$ .
9:       Update parameters via BPTT.
10:    end for
11:  end for
12: end while
```

- **Optimization Techniques:**

- Gradient clipping to prevent exploding gradients.
- Proper weight initialization.
- Truncated backpropagation through time.

4 Fast Weight Programmers (1992)

4.1 Theoretical Foundation

Fast Weight Programmers [Schmidhuber, 1992] introduce a novel architecture where one neural network (the programmer) rapidly modifies the weights of another network (the fast weight network). This enables dynamic adaptation to input patterns within the feed-forward pass.

4.2 Mathematical Framework

Definition 3 (Fast Weight System). *A fast weight system $\mathcal{F}$ is defined as:*

$$\mathcal{F} = (W_s, W_f, P, \phi),$$

where:

- W_s : Slow weights (traditional learned parameters).
- W_f : Fast weights (rapidly modified).
- P : Programming network.
- ϕ : Update rule for fast weights.

Theorem 3 (Fast Weight Convergence). *For a bounded fast weight update rule ϕ , the system converges to a stable state:*

$$\lim_{t \rightarrow \infty} \|W_f^t - W_f^*\|_F = 0,$$

where W_f^* is the optimal fast weight configuration.

Proof. 1. The fast weight update at time t is:

$$W_f^t = \phi(W_f^{t-1}, P(x_t)).$$

2. By the contraction mapping principle:

$$\|\phi(W_f^{t-1}, P(x_t)) - \phi(W_f^*, P(x_t))\|_F \leq \gamma \|W_f^{t-1} - W_f^*\|_F.$$

3. If $\gamma < 1$, then:

$$\|W_f^t - W_f^*\|_F \leq \gamma^t \|W_f^0 - W_f^*\|_F.$$

4. As $t \rightarrow \infty$, $\gamma^t \rightarrow 0$, ensuring convergence. □

Lemma 1 (Programming Network Stability). *The programming network output remains bounded:*

$$\|P(x_t)\| \leq L_p \|x_t\|,$$

where L_p is the Lipschitz constant of the programming network.

4.3 Implementation Details

- **Training Algorithm:**

- 1: Initialize slow weights W_s and programming network P .
- 2: **while** not converged **do**
- 3: **for** each input sequence $(x_1, \dots, x_T)$ **do**
- 4: Initialize fast weights W_f^0 .
- 5: **for** each time step t **do**
- 6: Generate weight updates: $\Delta W_f = P(x_t)$.
- 7: Update fast weights: $W_f^t = \phi(W_f^{t-1}, \Delta W_f)$.
- 8: Compute output: $y_t = f(W_s x_t + W_f^t x_t)$.
- 9: Compute loss: $\mathcal{L}(y_t, \hat{y}_t)$.
- 10: Update slow weights and programmer via backpropagation.
- 11: **end for**
- 12: **end for**
- 13: **end while**

- **Optimization Techniques:**

- Sparse fast weight updates.
- Memory-efficient weight generation.
- Regularization of programming network.
- Adaptive learning rates for different components.

5 Gated Neural Networks (1991)

5.1 Theoretical Foundation

Gated Neural Networks [Schmidhuber, 1991] introduce gating mechanisms to control information flow in neural networks. These gates allow the network to dynamically adjust the flow of information, enabling better modeling of sequential data.

5.2 Mathematical Framework

Definition 4 (Gated Network). *A gated network $\mathcal{G}$ is defined as:*

$$\mathcal{G} = (W, G, f),$$

where:

- W : Weight matrix for the main network.
- G : Gating mechanism, represented as a sigmoid function σ .
- f : Activation function (e.g., $\tanh$).

Theorem 4 (Gating Stability). *For a gated network with bounded weights, the output is stable:*

$$\|y_t\| \leq \frac{\|W\|}{1 - \|G\|},$$

where $\|G\| < 1$ ensures stability.

Proof. 1. The output at time t is given by:

$$y_t = f(Wx_t + Gy_{t-1}).$$

2. By the triangle inequality:

$$\|y_t\| \leq \|Wx_t\| + \|Gy_{t-1}\|.$$

3. If $\|G\| < 1$, the output is bounded as:

$$\|y_t\| \leq \frac{\|Wx_t\|}{1 - \|G\|}.$$

4. For bounded inputs $\|x_t\| \leq C$, the output remains stable.

□

5.3 Implementation Details

- **Training Algorithm:**

- 1: Initialize weights W and gating parameters G .
- 2: **while** not converged **do**
- 3: **for** each input sequence $(x_1, \dots, x_T)$ **do**
- 4: Initialize hidden state y_0 .
- 5: **for** each time step t **do**
- 6: Compute gated output: $y_t = f(Wx_t + Gy_{t-1})$.
- 7: Compute loss: $\mathcal{L}(y_t, \hat{y}_t)$.
- 8: Update weights: $W \leftarrow W - \eta \nabla_W \mathcal{L}$.
- 9: Update gating parameters: $G \leftarrow G - \eta \nabla_G \mathcal{L}$.
- 10: **end for**
- 11: **end for**
- 12: **end while**

- **Optimization Techniques:**

- Gradient clipping to prevent exploding gradients.
- Weight initialization (e.g., Xavier initialization).
- Regularization of gating parameters to ensure stability.

6 Long Short-Term Memory (LSTM) (1997)

6.1 Theoretical Foundation

Long Short-Term Memory (LSTM) [Hochreiter and Schmidhuber, 1997] is a type of RNN designed to address the vanishing gradient problem. It introduces memory cells and gating mechanisms to control information flow, enabling the learning of long-term dependencies.

6.2 Mathematical Framework

Definition 5 (LSTM System). *An LSTM system $\mathcal{L}$ is defined as:*

$$\mathcal{L} = (C, f, i, o, g),$$

where:

- C : Cell state, representing long-term memory.
- f : Forget gate, controlling memory retention.
- i : Input gate, controlling new information.
- o : Output gate, controlling output generation.
- g : Cell update function, typically $\tanh$.

Theorem 5 (LSTM Gradient Stability). *The gradient of the LSTM cell state is bounded:*

$$\left\| \frac{\partial C_t}{\partial C_{t-1}} \right\| \leq 1.$$

Proof. 1. The cell state update at time t is given by:

$$C_t = f_t \odot C_{t-1} + i_t \odot g_t.$$

2. The gradient of the cell state with respect to the previous cell state is:

$$\frac{\partial C_t}{\partial C_{t-1}} = f_t.$$

3. Since $f_t \in [0, 1]$, the gradient is bounded:

$$\left\| \frac{\partial C_t}{\partial C_{t-1}} \right\| \leq 1.$$

□

6.3 Implementation Details

- **Training Algorithm:**

1: Initialize LSTM parameters: W_f, W_i, W_o, W_g .

2: **while** not converged **do**

3: **for** each input sequence $(x_1, \dots, x_T)$ **do**

4: Initialize hidden state h_0 and cell state C_0 .

5: **for** each time step t **do**

6: Compute gates:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f), \quad i_t = \sigma(W_i[h_{t-1}, x_t] + b_i), \quad o_t = \sigma(W_o[h_{t-1}, x_t] + b_o).$$

7: Update cell state:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tanh(W_g[h_{t-1}, x_t] + b_g).$$

8: Compute hidden state:

$$h_t = o_t \odot \tanh(C_t).$$

9: Compute loss: $\mathcal{L}(h_t, \hat{y}_t)$.

10: Update parameters via backpropagation through time (BPTT).

11: **end for**

12: **end for**

13: **end while**

- **Optimization Techniques:**

- Gradient clipping to prevent exploding gradients.
- Dropout for regularization.
- Layer normalization for stable training.

7 LSTM with Forget Gates (2000)

7.1 Theoretical Foundation

LSTM with Forget Gates [Gers et al., 2000] enhances the original LSTM by introducing a forget gate, enabling better control over memory retention. This allows the network to learn when to reset or retain information.

7.2 Mathematical Framework

Definition 6 (LSTM with Forget Gates). *An LSTM system $\mathcal{L}$ is defined as:*

$$\mathcal{L} = (C, f, i, o, g, \phi),$$

where:

- C : Cell state.
- f : Forget gate.
- i : Input gate.
- o : Output gate.
- g : Cell update function.
- ϕ : Forget gate activation function.

Theorem 6 (Forget Gate Stability). *The forget gate ensures stable memory retention:*

$$\|C_t\| \leq \|f_t\| \|C_{t-1}\| + \|i_t\| \|g_t\|.$$

Proof. 1. The cell state update at time t is given by:

$$C_t = f_t \odot C_{t-1} + i_t \odot g_t.$$

2. By the triangle inequality:

$$\|C_t\| \leq \|f_t\| \|C_{t-1}\| + \|i_t\| \|g_t\|.$$

3. If $\|f_t\| < 1$, the memory is stable.

□

7.3 Implementation Details

- **Training Algorithm:**

- 1: Initialize LSTM parameters: W_f, W_i, W_o, W_g .
- 2: **while** not converged **do**
- 3: **for** each input sequence $(x_1, \dots, x_T)$ **do**
- 4: Initialize hidden state h_0 and cell state C_0 .

```

5:   for each time step  $t$  do
6:     Compute gates:
        
$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f), \quad i_t = \sigma(W_i[h_{t-1}, x_t] + b_i), \quad o_t = \sigma(W_o[h_{t-1}, x_t] + b_o).$$

7:     Update cell state:
        
$$C_t = f_t \odot C_{t-1} + i_t \odot \tanh(W_g[h_{t-1}, x_t] + b_g).$$

8:     Compute hidden state:
        
$$h_t = o_t \odot \tanh(C_t).$$

9:     Compute loss:  $\mathcal{L}(h_t, \hat{y}_t)$ .
10:    Update parameters via BPTT.
11:  end for
12:  end for
13: end while

```

- **Optimization Techniques:**

- Gradient clipping.
- Dropout.
- Layer normalization.

8 Gödel Machine (2003-2007)

8.1 Theoretical Foundation

The Gödel Machine [Schmidhuber, 2006, 2007] is a self-referential learning system that can provably improve any part of its own code, including its learning algorithm. It combines aspects of algorithmic probability theory and theorem proving to achieve provable self-improvement through mathematical reasoning about its own behaviors.

8.2 Mathematical Framework

Definition 7 (Gödel Machine System). *A Gödel Machine $\mathcal{G}$ is defined as:*

$$\mathcal{G} = (P, U, S, T, \mathcal{L}),$$

where:

- P : Proof searcher, operating on formal logical statements.
- U : Utility function measuring the machine's performance.
- S : Current software including all changeable parts.

- T : Theorem prover for verifying improvements.
- $\mathcal{L}$: Formal language for expressing proofs.

Theorem 7 (Self-Improvement Optimality). *For a Gödel Machine with utility function U , if there exists a self-modification that provably improves U , the machine will eventually find and implement it:*

$$\exists t : U(S_t) > U(S_0) \implies \exists \tau \leq t : S_\tau \neq S_0,$$

where S_t represents the system state at time t .

Proof. 1. The proof searcher continuously examines possible self-modifications:

$$\forall m \in \mathcal{M} : P(m) \implies U(S_m) > U(S_{\text{current}}).$$

2. By Gödel's completeness theorem, if a proof exists, it will be found:

$$\exists p : \text{Valid}(p) \wedge \text{Proves}(p, U(S_m) > U(S_{\text{current}})).$$

3. When found, the self-modification is implemented:

$$S_{\tau+1} = \text{Apply}(m, S_\tau).$$

4. The process guarantees eventual discovery of improving modifications. $\square$

Lemma 2 (Proof Search Completeness). *For any true statement in the formal system $\mathcal{L}$, the proof searcher will eventually find a proof if one exists:*

$$\forall \phi \in \mathcal{L} : (\exists p : \text{Proves}(p, \phi)) \implies \exists t : \text{Finds}(P, \phi, t).$$

8.3 Implementation Details

- **Training Algorithm:**

- 1: Initialize system S_0 with utility function U and proof searcher P .
- 2: **while** true **do**
- 3: // Main cycle of the Gödel Machine
- 4: **for** each candidate modification m **do**
- 5: Search for proof: $p = P(m)$
- 6: **if** $\text{Valid}(p) \wedge \text{Proves}(p, U(S_m) > U(S_{\text{current}}))$ **then**
- 7: Verify proof using theorem prover T
- 8: **if** verification successful **then**
- 9: Apply modification: $S_{\text{current}} = \text{Apply}(m, S_{\text{current}})$
- 10: **break** // Start new cycle with improved system
- 11: **end if**
- 12: **end if**
- 13: **end for**

14: Execute regular problem-solving subroutine
15: **end while**

- **Optimization Techniques:**

- Bias proof search toward simple modifications first.
- Cache intermediate proof results.
- Use incremental proof verification.
- Implement resource bounds for proof search.

9 Clockwork RNNs (2014)

9.1 Theoretical Foundation

Clockwork RNNs [Koutnik et al., 2014] introduce a multi-scale architecture where different parts of the network operate at different temporal frequencies. This design allows the network to capture both short-term and long-term dependencies efficiently.

9.2 Mathematical Framework

Definition 8 (Clockwork RNN). *A Clockwork RNN $\mathcal{C}$ is defined as:*

$$\mathcal{C} = (H, T, f),$$

where:

- H : Hidden states, partitioned into K modules.
- T : Temporal scales or frequencies, with each module updating only at specified time intervals.
- f : Activation function.

Theorem 8 (Multi-Scale Stability). *For a Clockwork RNN with bounded weights, the hidden states are stable:*

$$\|h_t\| \leq \frac{\|W\|}{1 - \|T\|},$$

assuming $\|T\| < 1$ and bounded inputs.

Proof. 1. The hidden state update at time t is given by:

$$h_t = f(Wx_t + Th_{t-1}),$$

where certain blocks of h_{t-1} may remain unchanged depending on the time step and module frequency.

2. By the triangle inequality:

$$\|h_t\| \leq \|Wx_t\| + \|Th_{t-1}\|.$$

3. If $\|T\| < 1$, the hidden states are bounded:

$$\|h_t\| \leq \frac{\|Wx_t\|}{1 - \|T\|}.$$

□

9.3 Implementation Details

- **Training Algorithm:**

- 1: Initialize Clockwork RNN parameters: W, T .
- 2: **while** not converged **do**
- 3: **for** each input sequence $(x_1, \dots, x_T)$ **do**
- 4: Initialize hidden states h_0 .
- 5: **for** each time step t **do**
- 6: Update hidden states:

$$h_t = f(Wx_t + Th_{t-1}),$$

where only the modules scheduled to update at step t are computed.

- 7: Compute loss: $\mathcal{L}(h_t, \hat{y}_t)$.
- 8: Update parameters via backpropagation.
- 9: **end for**
- 10: **end for**
- 11: **end while**

- **Optimization Techniques:**

- Multi-scale gradient updates.
- Temporal regularization.
- Sparse connections for efficiency.

10 Highway Networks (2015)

10.1 Theoretical Foundation

Highway Networks [Srivastava et al., 2015] introduce gating mechanisms to enable training of very deep networks. These gates allow information to flow directly through the network, mitigating the vanishing gradient problem.

10.2 Mathematical Framework

Definition 9 (Highway Network). *A Highway Network $\mathcal{H}$ is defined as:*

$$\mathcal{H} = (W, G, f),$$

where:

- W : Weight matrix.
- G : Gating mechanism, represented as a sigmoid function σ .
- f : Activation function.

Theorem 9 (Highway Network Stability). *For a Highway Network with bounded weights, the output is stable:*

$$\|y_t\| \leq \frac{\|W\|}{1 - \|G\|},$$

assuming $\|G\| < 1$.

Proof. 1. The output at time t is given by:

$$y_t = f(Wx_t + Gy_{t-1}).$$

2. By the triangle inequality:

$$\|y_t\| \leq \|Wx_t\| + \|Gy_{t-1}\|.$$

3. If $\|G\| < 1$, the output is bounded:

$$\|y_t\| \leq \frac{\|Wx_t\|}{1 - \|G\|}.$$

□

10.3 Implementation Details

• Training Algorithm:

- 1: Initialize Highway Network parameters: W, G .
- 2: **while** not converged **do**
- 3: **for** each input or layer x_t **do**
- 4: Compute gated output:

$$y_t = f(Wx_t + Gy_{t-1}).$$

- 5: Compute loss: $\mathcal{L}(y_t, \hat{y}_t)$.
- 6: Update parameters via backpropagation.
- 7: **end for**

8: **end while**

- **Optimization Techniques:**

- Gradient clipping.
- Weight initialization.
- Gating regularization.

11 PowerPlay (2011)

11.1 Theoretical Foundation

PowerPlay [Schmidhuber, 2011] is a framework for self-motivated AI that learns to solve increasingly complex problems through intrinsic motivation. It continually searches for the simplest unsolvable problem and solves it.

11.2 Mathematical Framework

Definition 10 (PowerPlay System). *A PowerPlay system $\mathcal{P}$ is defined as:*

$$\mathcal{P} = (T, S, \phi),$$

where:

- T : Task space, representing a set of problems.
- S : Solver, represented as a parameterized function S_θ .
- ϕ : Update rule for the solver.

Theorem 10 (Task Complexity Convergence). *For a bounded update rule ϕ , the system converges to solving increasingly complex tasks:*

$$\lim_{t \rightarrow \infty} \|T_t - T^*\|_F = 0,$$

where T^* is the optimal task space (in the sense that it encompasses all solvable tasks within a given complexity bound).

Proof. 1. The task update at time t is given by:

$$T_t = \phi(T_{t-1}, S_{t-1}).$$

2. By the contraction mapping theorem, if ϕ is a contraction mapping with Lipschitz constant $\gamma < 1$, then:

$$\|\phi(T_{t-1}, S_{t-1}) - \phi(T^*, S^*)\|_F \leq \gamma \|T_{t-1} - T^*\|_F.$$

3. Iterating this inequality yields:

$$\|T_t - T^*\|_F \leq \gamma^t \|T_0 - T^*\|_F.$$

4. As $t \rightarrow \infty$, $\gamma^t \rightarrow 0$, and thus $\|T_t - T^*\|_F \rightarrow 0$.

□

11.3 Implementation Details

- **Training Algorithm:**

- 1: Initialize solver S_0 and task space T_0 .
- 2: **while** not converged **do**
- 3: **for** each task $\mathcal{T}_i$ in T_t **do**
- 4: Evaluate performance of S_t on $\mathcal{T}_i$.
- 5: Compute meta-gradient: $\nabla_{\phi} \mathcal{L}(S_t, \mathcal{T}_i)$.
- 6: Update solver: $S_t = \phi(S_{t-1}, T_{t-1})$.
- 7: Update task space: $T_t = T_{t-1} + \eta \nabla_T \mathcal{L}$.
- 8: **end for**
- 9: **end while**

- **Optimization Techniques:**

- Intrinsic motivation for task selection.
- Curriculum learning to prioritize simpler tasks first.
- Regularization to prevent overfitting to trivial tasks.

12 Self-Referential Meta-Learning

The recent work by Kirsch and Schmidhuber [2022] advances the concept of self-referential architectures, emphasizing systems that autonomously modify their own computational structures and parameters. These architectures, which are capable of iterative self-improvement, build upon Schmidhuber’s earlier theories [1987, 1993]. The key innovation lies in the ability of these systems to internalize the process of meta-learning without reliance on external optimization algorithms.

12.1 Core Definitions

Let a self-referential neural architecture be defined as a tuple:

$$(\phi, g, x) \tag{1}$$

where:

- $\phi = \{\theta, h, y\}$ represents all variables in the system
- θ : network parameters (weights)
- h : time-varying variables (memory/hidden states)
- y : outputs
- g : the network function that updates all variables
- x : external inputs

The key update equation is:

$$\phi_{t+1} \leftarrow g_{\phi}(x_t) \tag{2}$$

12.2 Necessary Conditions

Theorem 11 (Variable Sharing). *For a connected computational graph with $|\phi|$ variables, some variables must be reused in multiple operations to generate ϕ_{t+1} from ϕ_t .*

Proof. 1. $|\phi_{t+1}| = |\phi_t| = |\phi|$

2. Any operation generating an element in ϕ_{t+1} that uses more than one element from ϕ_t must reuse elements
3. Therefore, variable sharing is necessary for full self-modification

□

12.3 Implementation Framework

The practical implementation uses:

12.3.1 Outer Product Mechanism

$$W_t = W_{t-1} + \sigma(\beta_t)(\psi(v_t) - \psi(\bar{v}_t)) \otimes \psi(k_t) \quad (3)$$

where:

- W_t : weight matrix at time t
- β_t : learning rate
- $v_t, \bar{v}_t$: value vectors
- k_t : key vector
- ψ : activation function
- σ : sigmoid function
- $\otimes$: outer product

12.3.2 Key-Value Generation

$$[y_t, k_t, q_t, \beta_t] = W_{t-1}\psi(x_t) \quad (4)$$

$$\bar{v}_t = W_{t-1}\psi(k_t) \quad (5)$$

$$v_t = W_{t-1}\psi(q_t) \quad (6)$$

12.4 Theoretical Results

Theorem 12 (Representational Equivalence). *Any memory-based architecture $h, y \leftarrow f_\theta(h, x)$ can be represented by a self-referential architecture $\phi, y \leftarrow g_\phi(x)$ and vice versa.*

Proof sketch. 1. Memory architecture can be emulated by storing variables in self-referential network’s memory

2. Self-referential updates can be emulated by memory architecture with sufficient capacity

□

12.5 Key Properties

1. All variables are subject to modification by the network itself
2. Updates require variable sharing through outer products or similar mechanisms
3. System can learn:
 - Task-specific behaviors
 - Learning algorithms
 - Meta-learning strategies
 - Meta-meta-learning capabilities

12.6 Optimization Framework

The system optimizes through Fitness Monotonic Execution (FME):

$$p(\phi) \propto \exp(R(\phi)/\Delta t) \tag{7}$$

where:

- $p(\phi)$: probability of selecting solution ϕ
- $R(\phi)$: average reward of solution ϕ
- Δt : solution lifetime

13 Conclusion

We have surveyed foundational and influential neural network and meta-learning architectures by Jürgen Schmidhuber, spanning over three decades of innovation. Beginning with early self-referential learning systems, we traced the evolution of mechanisms for processing sequences, managing weights dynamically, and enabling self-improvement, culminating in the PowerPlay framework that

addresses continually expanding sets of tasks. Each architecture introduced novel principles for stability, convergence, and efficiency, paving the way for modern deep learning systems that rely on similar mechanisms of gating, memory, and meta-learning. Beyond their historical importance, these architectures continue to inspire research in sequence modeling, lifelong learning, and robust autonomous systems. They serve as milestones that illustrate how iterative self-improvement, clever gating, and principled handling of temporal structure can significantly advance the capabilities of neural-based models.

References

- Felix A Gers, Jürgen Schmidhuber, and Fred Cummins. Learning to forget: Continual prediction with lstm. *Neural computation*, 12(10):2451–2471, 2000.
- Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- Louis Kirsch and Jürgen Schmidhuber. Self-referential meta learning. *AutoML Conference Workshop Track*, 2022.
- Jan Koutník, Klaus Greff, Faustino Gomez, and Jürgen Schmidhuber. A clock-work rnn. *arXiv preprint arXiv:1402.3511*, 2014.
- Juergen Schmidhuber. Goedel machines: Self-referential universal problem solvers making provably optimal self-improvements, 2006. URL <https://arxiv.org/abs/cs/0309048>.
- Jürgen Schmidhuber. Making the world differentiable: On using fully recurrent self-supervised neural networks for dynamic reinforcement learning and planning in non-stationary environments. *Technical Report FKI-126-90, Institut für Informatik, Technische Universität München*, 1990.
- Jürgen Schmidhuber. Learning to control fast-weight memories: An alternative to dynamic recurrent networks. *Neural Computation*, 4(1):131–139, 1992.
- Jürgen Schmidhuber. Evolutionary principles in self-referential learning. *Diploma thesis, Technische Universität München*, 1987.
- Jürgen Schmidhuber. Neural sequence chunkers. *Technical Report FKI-148-91*, 1991.
- Jürgen Schmidhuber. Self-referential learning. *Neural Networks*, 6(1):101–108, 1993.
- Jürgen Schmidhuber. *Gödel Machines: Fully Self-referential Optimal Universal Self-improvers*, volume 8, pages 199–226. 01 2007. ISBN 978-3-540-23733-4. doi: 10.1007/978-3-540-68677-4_7.

Jürgen Schmidhuber. Powerplay: Training an increasingly general problem solver by continually searching for the simplest still unsolvable problem. *Frontiers in Psychology*, 2:313, 2011.

Rupesh K Srivastava, Klaus Greff, and Jürgen Schmidhuber. Highway networks. *arXiv preprint arXiv:1505.00387*, 2015.